



Guia de Integração
Gateway de Pagamentos Reduniq

Versão 7.5

Índice

Introdução.....	3
Contactos.....	3
Integração com a aplicação do comerciante.....	4
Inicializar pagamento web.....	10
Redirecionar para a página do Gateway de Pagamentos	10
Consultar os dados de um pagamento	10
Ligação ao gateway.....	11
Requisitos de segurança	11
Ambiente de testes.....	11
Ambiente de produção	11
Funções da API	12
initPayment	12
getResult	18
doCapture	20
doRefund.....	22
doVoid.....	23
searchTransactions	24
createPaymentToken	26
disablePaymentToken.....	28
createRecurringPayment.....	29
getRecurringPayment	31
disableRecurringPayment	33
doPaymentToken.....	35
doPaymentCard	38
doPayByLink.....	46
resultPayByLink	49
voidPayByLink.....	50
regenPayByLink.....	51
ANEXOS.....	53

Introdução

Este documento descreve o procedimento de integração do *Gateway de Pagamentos* da Reduniq com a aplicação do comerciante.

A interface *web* do *Gateway de Pagamentos* permite ao comerciante disponibilizar aos seus clientes um meio para efetuar pagamentos seguros, utilizando a solução de pagamentos VISA, Mastercard, AMEX, PayPal, MB WAY ou Pagamento de serviços (MB).

Contactos

A Reduniq disponibiliza os seguintes contactos para dar suporte à integração do *Gateway de Pagamentos*:

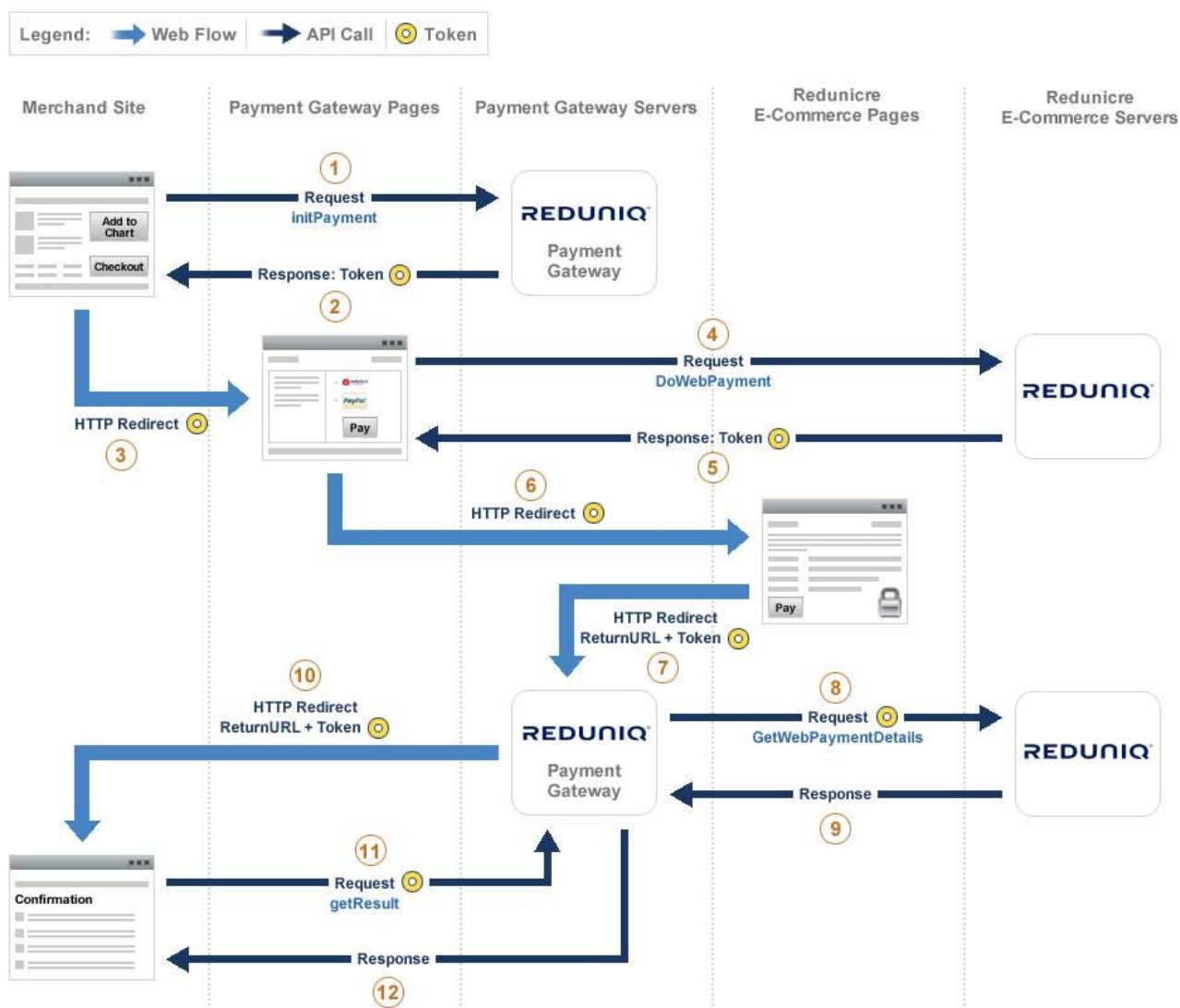
Tel.: 21 313 29 00

Fax: 21 313 29 30

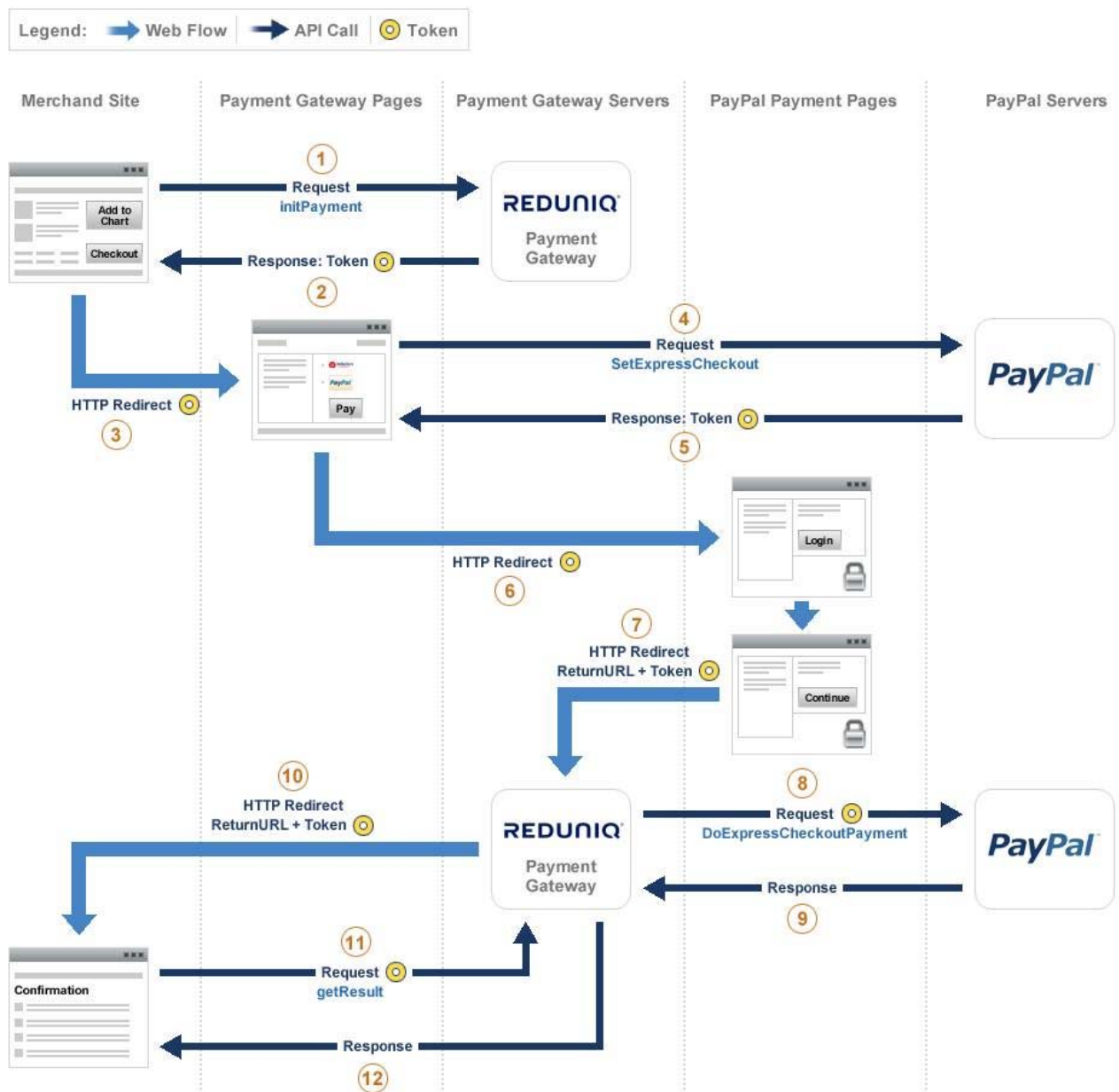
E-mail: apoiotecnico@unicre.pt

Integração com a aplicação do comerciante

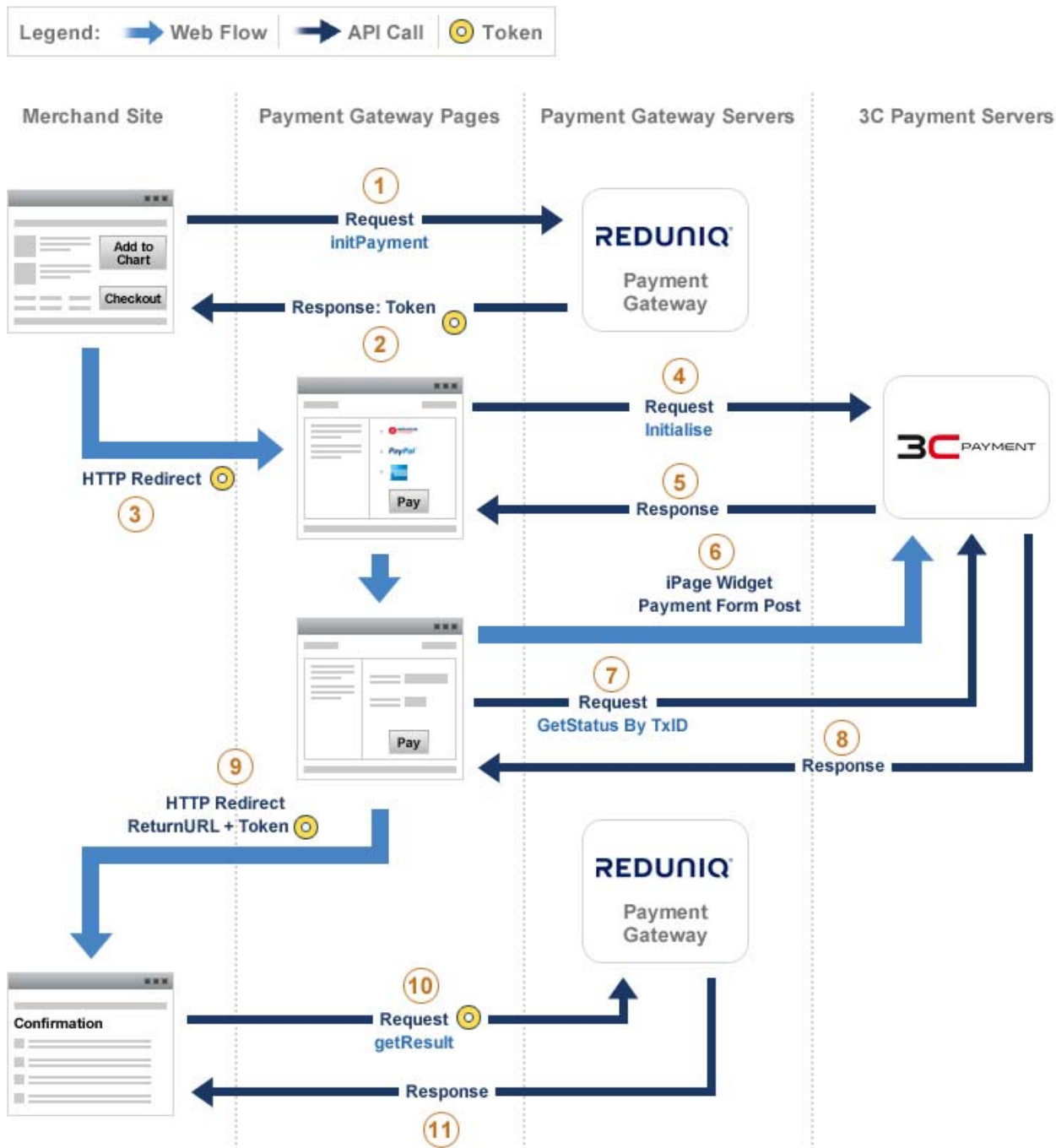
A integração da aplicação de e-commerce com o *Gateway de Pagamentos* da Reduniq disponibiliza, aos clientes do comerciante, modos de pagamentos seguros através do Reduniq E-Commerce, PayPal, AMEX (3C iPage), MB WAY (SIBS), VISA/Mastercard (DPG), MB WAY (DPG) ou Pagamento de Serviços/MB (DPG).



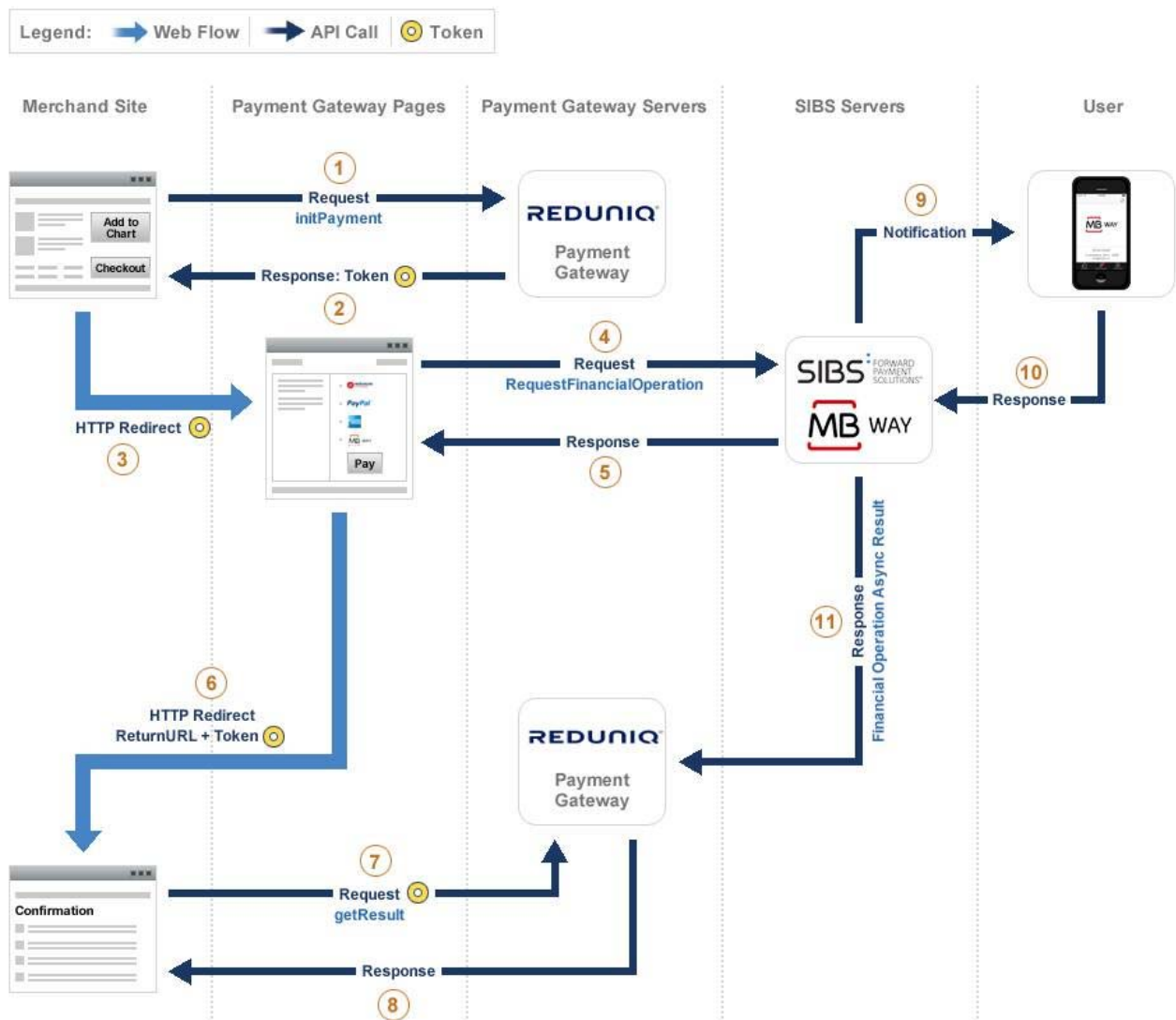
Workflow do processo de pagamento (Reduniq E-Commerce)



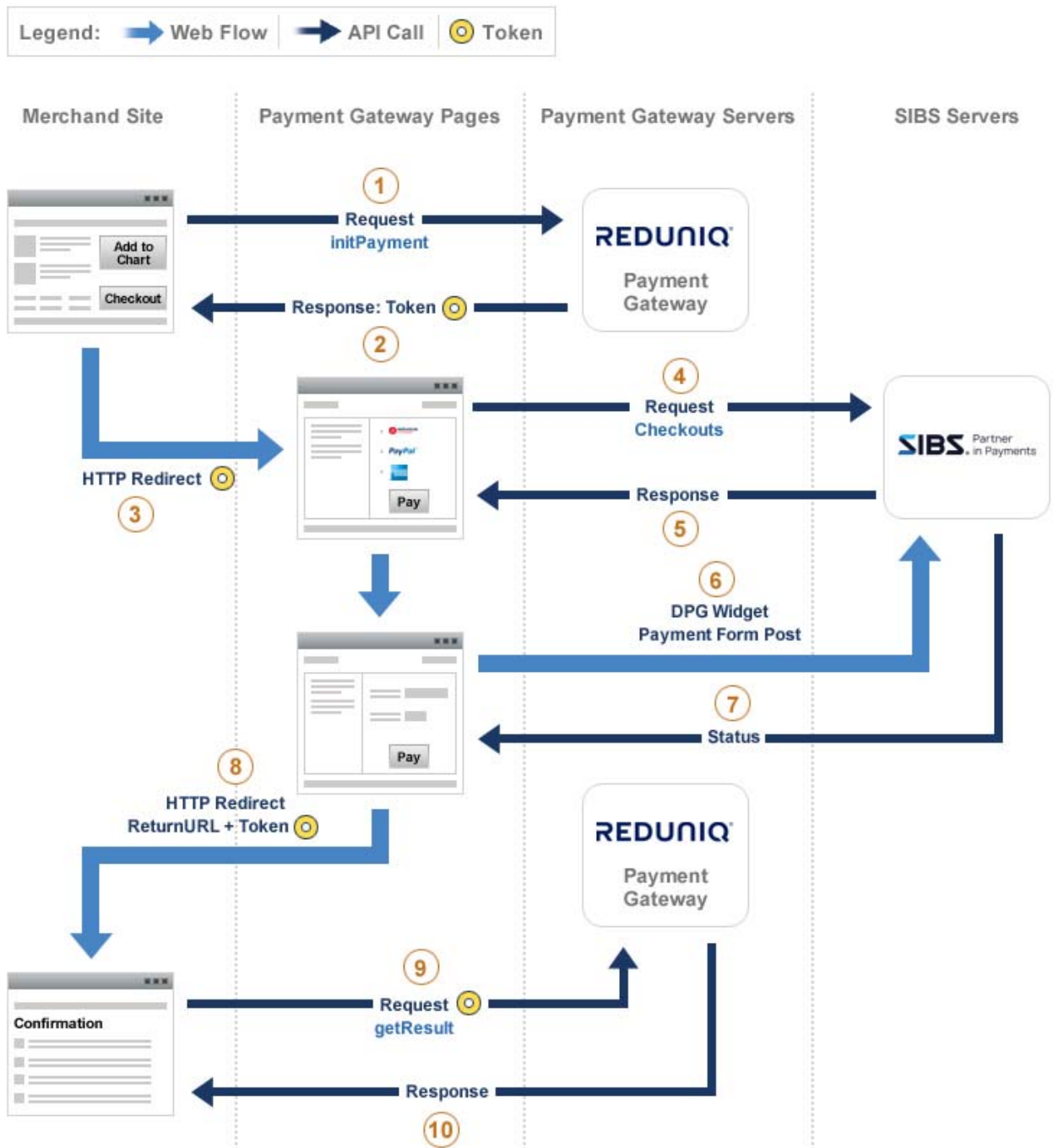
Workflow do processo de pagamento (PayPal)



Workflow do processo de pagamento (AMEX 3C iPage)



Workflow do processo de pagamento (MB WAY SIBS)



Workflow do processo de pagamento (VISA/Mastercard/MB WAY/MB DPG)

Descrição do processo:

- O site do comerciante chama a função “initPayment” para iniciar o processo de pagamento. Na chamada ao serviço, para além dos seus dados de identificação (*username* e *password*), deve indicar os restantes dados necessários para iniciar o processo (nome do cliente, valor a pagar, etc.). A *password* não é armazenada no *gateway* e serve de chave de descriptação dos dados de acesso às plataformas de pagamentos disponíveis para esse comerciante.
- O cliente é redirecionado para a página *web* do *gateway* onde este seleciona a plataforma de pagamentos a utilizar. Se o comerciante apenas dispõe uma solução de pagamento ou caso defina o meio de pagamento a usar no “initPayment”, a página de seleção pode ou não ser exibida ao cliente.
- O *Gateway de Pagamentos* estabelece as comunicações necessárias com a plataforma selecionada.
- O cliente é redirecionado para a plataforma selecionada onde este efetuará o pagamento indicando os dados do seu cartão VISA/MASTERCARD/AMEX ou utilizando a sua conta PayPal.
- Após conclusão (ou cancelamento) do pagamento, a plataforma de pagamento utilizada redireciona para o *Gateway de Pagamentos* que regista em *log* toda a informação associada.
- A página de retorno do *gateway* chama o *webservice* disponibilizado pela plataforma selecionada para obter a informação associada ao processo e envia os dados relevantes na resposta à aplicação do comerciante.
- O cliente é redirecionado para a página do comerciante onde será exibida a mensagem de sucesso/erro.
- No caso do comerciante não receber o retorno num determinado período de tempo, deverá chamar a função “getResult” para verificar o estado do mesmo. Poderá também consultar o backoffice próprio da plataforma de pagamento utilizada.

Inicializar pagamento web

A inicialização de um pagamento *web* (função *initPayment*), consiste na comunicação ao *Gateway de Pagamentos* de toda a informação necessária para se proceder a um pagamento.

Os campos obrigatórios para iniciar um pagamento *web* estão indicados em “Funções da API”.

Redirecionar para a página do Gateway de Pagamentos

Uma vez iniciado um pagamento, o cliente deve ser redirecionado para a página web do *Gateway de Pagamentos*. Neste processo estão envolvidas as seguintes operações:

- Na inicialização do pagamento, obter o *token* de sessão e o endereço da página *web* do *Gateway de Pagamentos*;
- Associar, no sistema de informação do comerciante, o *token* com a compra;
- Redirecionar o cliente para a página *web* do gateway, através de uma resposta HTTP.

Consultar os dados de um pagamento

Quando o processo de pagamento termina, o *Gateway de Pagamentos* redireciona o cliente para a página do comerciante.

A aplicação do comerciante deve chamar sempre a função “*getResult*” para obter os detalhes do pagamento, nomeadamente, o código de resposta e estado da transação. Com essa informação o comerciante pode informar o seu cliente qual foi o resultado do pagamento (consultar a tabela em anexo “Códigos e mensagens”).

Ligação ao *gateway*

Requisitos de segurança

Para garantir que todas as comunicações com o *Gateway de Pagamentos* são seguras deve garantir que:

- Todas as ligações são feitas com HTTPS/SSL;
- Nunca divulgue os seus dados de autenticação (API *username* e API *password*);
- Verifique que o certificado do servidor presente na ligação HTTPS pertence à Reduniq e que este é válido (não expirou nem foi revogado).

Ambiente de testes

Para testar a integração da aplicação do comerciante com o *Gateway de Pagamentos* Reduniq, utilize os seguintes endereços:

REST API:

<https://pagamentos.sandbox.reduniq.pt/api-gateway/v7.0/rest/>

Nota: Todos os pedidos devem ser submetidos através do método POST

Interface *web* para testar os métodos disponíveis:

<https://pagamentos.sandbox.reduniq.pt/test-gateway/>

Ambiente de produção

Em produção, utilize os endereços:

REST API:

<https://pagamentos.reduniq.pt/api-gateway/v7.0/rest/>

Nota: Todos os pedidos devem ser submetidos através do método POST

Funções da API

O *Gateway de Pagamentos* Reduniq é disponibilizado no formato REST. Todos os dados enviados devem ser codificados em Unicode (UTF-8). Nas respostas, a API retorna sempre os dados em UTF-8.

A API disponibiliza as seguintes funções:

Função	Descrição
initPayment	Inicialização de um pagamento <i>web</i>
getResult	Obter os detalhes de um pagamento
doCapture	Aceitar um pagamento autorizado
doRefund	Reembolsar um pagamento autorizado e aceite
doVoid	Cancelar um pagamento autorizado
searchTransactions	Pesquisar transações
createPaymentToken	Criar um <i>token</i> de pagamento
disablePaymentToken	Desativar um <i>token</i> de pagamento
createRecurringPayment	Criar um pagamento recorrente
getRecurringPayment	Obter os detalhes de um pagamento recorrente
disableRecurringPayment	Desativar um pagamento recorrente
doPaymentToken	Efetuar um pagamento com recurso a um <i>token</i> de pagamento
doPaymentCard	Efetuar um pagamento com cartão
doPayByLink	Gerar um novo pedido de pagamento
resultPayByLink	Obter o estado de um determinado pedido de pagamento
voidPayByLink	Cancelar um pedido de pagamento
regenPayByLink	Renovar um pedido de pagamento

initPayment

A função “*initPayment*” inicializa um pagamento antes de redirecionar o cliente para a página *web* do *Gateway de Pagamentos*, onde este selecionará o meio de pagamento.

Estrutura de dados do pedido:

Campo	Descrição	Obrig.	Formato	Observações
method	Nome da função	sim	AN50	initPayment
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payment.amount	Valor total a pagar (order amount + order taxes + shipping amount)	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	sim	N3	100 – Autorização 101 – Autorização + Captura
payment.solution	Código da solução de pagamento a utilizar. Caso não seja indicada será apresentada a lista de meios de pagamento disponíveis para este comerciante.	não	N3	100 - REDUNIQ E-Commerce 101 - PayPal (SOAP) 102 - AMEX (3C iPage) 105 - PayPal (REST) 106 - Cartão de crédito (DPG) 107 - MB WAY (DPG) 108 - Pagamento de Serviços/MB (DPG) 109 - Cartão de crédito (SPG) 110 - MB WAY (SPG)

				111 - Pagamento de Serviços/MB (SPG) 112 – Parcela Já 113 – Cartão de crédito (Cybersource) 114 – Google Pay (Cybersource) 115 – Apple Pay (Cybersource) 116 – PIX (Braza) 117 – Click to Pay (Cybersource) 118 – Samsung Pay (Cybersource)
payment.deadline	Data limite de pagamento	não	AN10 / AN19	2012-12-16 ou 2012-12-16 15:30:00 Para pagamentos assíncronos (p.e.: pagamento de serviços)
payment.description	Descrição do pagamento	sim	AN255	
order.ref	Referência do pagamento	sim	AN50	Cada pagamento deve ter uma referência única
order.amount	Valor a pagar (sem impostos e sem transporte)	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
order.taxes	Valor total dos impostos	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
order.date	Data e hora do pagamento	sim	AN19	2012-12-15 20:30:00
order.shipping	Expedição / entrega	não	N1	0 – Não (por omissão) 1 – Sim Se existe expedição e o order.details não é indicado, é considerado um bem “Físico/Material”
order.details	Detalhes do pagamento	não		Ver estrutura de dados “order.details”
buyer.firstName	Nome do cliente	não	AN50	
buyer.lastName	Sobrenome do cliente	não	AN50	
buyer.email	E-mail do cliente	não	AN150	
buyer.phone	Telefone do cliente	não	AN15	Exemplos: 910000000 00351910000000 +351910000000
buyer.shipping.name	Nome da pessoa associada a este endereço	não	AN32	
buyer.shipping.street1	Expedição: Rua 1	não	AN100	
buyer.shipping.street2	Expedição: Rua 2	não	AN100	
buyer.shipping.city	Expedição: Cidade	não	AN40	
buyer.shipping.state	Província/estado	não	AN40	
buyer.shipping.zipCode	Expedição: Código postal	não	AN20	
buyer.shipping.country	Expedição: País	não	AN2	ISO 3166-1 (exemplo: “pt”)
buyer.shipping.phone	Expedição: Telefone	não	AN15	
buyer.shipping.amount	Expedição: Valor	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€ Quando definido, o order.shipping deve ser igual a “1”
buyer.billing.street1	Faturação: Rua 1	não	AN100	

buyer.billing.street2	Faturação: Rua 2	não	AN100	
buyer.billing.city	Faturação: Cidade	não	AN40	
buyer.billing.state	Província/estado	não	AN40	
buyer.billing.zipCode	Faturação: Código postal	não	AN20	
buyer.billing.country	Faturação: País	não	AN2	ISO 3166-1 (exemplo: "pt")
buyer.billing.tin	Número de identificação fiscal	não	AN20	
payToken.action	Código para criação de um <i>token</i> de pagamento	não	N3	500 – Criar <i>token</i> de pagamento
payToken.shared	Partilha do token	não	N1	0 – token não partilhado 1 – token partilhado
mode	Modo de integração	não	AN10	Valores possíveis: <i>redirect</i> (por omissão), <i>lightbox</i> , <i>iframe</i> ou <i>webless</i>
returnUrlOk	URL de retorno para a página do comerciante	sim	AN255	Usado quando o pagamento é aceite.
returnUrlError	URL de retorno para a página do comerciante	sim	AN255	Usado quando o pagamento é recusado ou cancelado.
returnUrlTarget	Destino do retorno (para o modo <i>lightbox</i>)	não	AN6	Valores possíveis: <i>top</i> (por omissão) ou <i>self</i>
notificationUrl	URL de notificação	não	AN255	
targetOriginUrl	URL de origem (loja do comerciante)	não	AN255	Importante para os modos <i>lightbox</i> ou <i>iframe</i>
privateData	Dados privados	não		Ver estrutura de dados "privateData"
languageCode	Código do idioma	não	AN6	ISO 639-2 (exemplo: "por")

Estrutura de dados "order.details" (0 a 100)

Campo	Descrição	Obrig.	Formato	Observações
name	Nome/código do produto ou serviço	sim	AN50	
amount	Valor unitário (sem impostos)	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
tax	Valor unitário do imposto	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
quantity	Quantidade	não	N5	
category	Categoria (Físico/Digital)	não	N1	0 – Digital/Serviço (por omissão) 1 – Físico/Material

Estrutura de dados "privateData" (0 a 100)

Campo	Descrição	Obrigatório	Formato	Observações
name	Nome/chave	sim	N50	Exemplo: "NIF"
value	Valor	sim	N255	Exemplo: "508855567"

Exemplo de um pedido em formato REST:

```
{
  "method": "initPayment",
  "api": {
    "username": "demouser",
```

```

    "password": "abc123"
  },
  "payment": {
    "amount": 700,
    "action": 100,
    "solution": "",
    "deadline": "",
    "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
  },
  "order": {
    "ref": "ref1680534950",
    "amount": 500,
    "taxes": 100,
    "date": "2023-04-03 16:15:42",
    "shipping": "1",
    "details": [
      {
        "name": "Item #1",
        "amount": 250,
        "tax": 50,
        "quantity": 1,
        "category": "1"
      },
      {
        "name": "Item #2",
        "amount": 250,
        "tax": 50,
        "quantity": 1,
        "category": "1"
      }
    ]
  },
  "buyer": {
    "firstName": "Manuel",
    "lastName": "Silva",
    "email": "manuel.silva@mail.com",
    "phone": "",
    "shipping": {
      "name": "Manuel Silva",
      "street1": "Rua Alberto Sampaio",
      "street2": "n50, 2D",
      "city": "Lisboa",
      "state": "Lisboa",
      "zipCode": "2100-456",
      "country": "pt",
      "phone": "+351213456784",
      "amount": 100
    }
  },
  "billing": {
    "street1": "Rua Alberto Sampaio",
    "street2": "n50, 2D",
    "city": "Lisboa",
    "state": "Lisboa",
    "zipCode": "2100-456",

```

```

    "country": "pt"
  },
  "payToken": {
    "action": 500,
    "shared": 1
  },
  "returnUrlOk": " https://www.merchant.pt/ok",
  "returnUrlError": " https://www.merchant.pt/error",
  "notificationUrl": " https://www.merchant.pt/notification",
  "privateData": [
    {
      "name": "key #1",
      "value": "value #1"
    },
    {
      "name": "key #2",
      "value": "value #2"
    },
    {
      "name": "key #3",
      "value": "value #3"
    }
  ],
  "languageCode": "por"
}

```

Estrutura de dados da Resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 00000000 - OK	AN11	Ver em anexo “Códigos e mensagens” Em modo <i>webless</i> o código do resultado é diferente de 00000000. É necessário validar o valor de <i>transaction.status</i>
result.message	Mensagem do resultado	AN255	
transaction.id	ID único da transação	AN50	
transaction.date	Data e hora da transação	AN19	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	0 – Não iniciada 2 – Em curso 3 – Erro / terminada
transaction.extraData	Informação adicional associada à transação		Dados disponíveis em modo <i>webless</i> Por exemplo, para pagamentos de serviços por MB, é devolvida a entidade, referência, valor e data limite de pagamento. Ver estrutura de dados “ <i>transaction.extraData</i> ”
token	<i>token</i> atribuído ao pagamento	AN50	

redirectUrl	URL para redirecionar o cliente para a página de pagamento	AN255	Vazio em modo <i>webless</i>
urlPath	Caminho do URL utilizado pelo <i>widget</i> (modo <i>lightbox</i>)	AN255	Vazio em modo <i>webless</i>

Estrutura de dados “transaction.extraData” (0 a N)

Campo	Descrição	Obrigatório	Formato	Observações
name	Nome/chave	sim	N50	Exemplo: “reference”
value	Valor	sim	N255	Exemplo: “962006246”

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "transaction": {
    "id": "",
    "status": "0",
    "date": "",
    "extraData": []
  },
  "token": "f34b971751d47fcb418dfd590d24c124",
  "redirectUrl": "https://www.xpto.com/token?lang=pt",
  "urlPath": "xxxx/token"
}
```

A função “getResult” permite ao comerciante consultar os detalhes e o resultado de um pagamento, indicando no pedido, o *token* associado ao pagamento.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	getResult
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
token	<i>token</i> atribuído ao pagamento	sim	AN50	

Exemplo de um pedido em formato REST:

```
{
  "method": "getResult",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "token": "01417b2b102624f00fcd9a0226820601"
}
```

Estrutura de dados da resposta:

Campo	Descrição	Form ato	Observações
result.code	Código do resultado: 10000000 – OK (RU E-Com) 20000000 – OK (PayPal) 30000000 – OK (AMEX / 3C iPage) 60000000 – OK (PayPal / REST) 5000000000 – OK (MB / DPG) 7000000000 – OK (CC / DPG) 8000000000 – OK (MB WAY / DPG) 9000000000 – OK (CC / SPG) 10000000000 – OK (MB WAY / SPG) 11000000000 – OK (MB / SPG) 12000000000 – OK (Parcela Já) 13000000000 – OK (CC / CS) 14000000000 – OK (G. Pay / CS) 15000000000 – OK (A. Pay / CS) 16000000000 – OK (PIX / Braza) 17000000000 – OK (CTP / CS) 18000000000 – OK (S.Pay / CS)	AN11	Ver em anexo “Códigos e mensagens” Nota: A avaliação do sucesso/insucesso de um pagamento deve ser efetuada através do campo <i>transaction.status</i>
result.message	Mensagem do resultado	AN25 5	
transaction.id	ID único da transação	AN50	
transaction.isFraud		AN1	Disponível para o Reduniq E-commerce 0 – risco de fraude não detetado 1 – risco de fraude

transaction.date	Data e hora da transação	AN19	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Aguarda seleção do meio de pagamento 2 – Em curso 3 – Erro / terminada 4 – Sucesso / terminada
transaction.extraData	Informação adicional associada à transação		Por exemplo, para pagamentos de serviços por MB, é devolvida a entidade, referência, valor e data limite de pagamento. Ver estrutura de dados "transaction.extraData"
payment.solution	Solução de pagamento selecionada pelo cliente	N3	100 – Reduniq E-Commerce 101 – PayPal 102 – AMEX / 3C iPage ...
payment.amount	Valor total do pagamento	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	N4	100 – Autorização 101 – Autorização + Captura
payToken.action	Código da ação do token de pagamento	N4	
payToken.id	Id do token de pagamento	N20	
payToken.sharedKey	Chave partilhada do token de pagamento	AN36	
privateData	Dados privados		Ver estrutura de dados "privateData"

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00800000",
    "message": "Waiting for confirmation of non-instant payment"
  },
  "transaction": {
    "id": "8ac7a4a1874921bd01874d4cfece65ff",
    "status": "2",
    "extraData": [
      {
        "name": "entity",
        "value": "28597"
      },
      {
        "name": "reference",
        "value": "962006246"
      },
      {
        "name": "amountValue",
        "value": "700"
      },
      {
        "name": "expireDate",
```

```

      "value": "2023-04-06 23:59:59"
    }
  ]
},
"payment": {
  "amount": "700",
  "action": "100",
  "solution": "108"
},
"payToken": {
  "id": "100005",
  "action": "500",
  "sharedKey": "d03430a3-64d1-42c8-8dbf-3535f0004d7f"
},
"privateData": [
  {
    "name": "key #3",
    "value": "value #3"
  },
  {
    "name": "key #2",
    "value": "value #2"
  },
  {
    "name": "key #1",
    "value": "value #1"
  }
]
}

```

doCapture

A função “doCapture” permite ao comerciante aceitar uma autorização de pagamento anteriormente validada.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	doCapture
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
transaction.id	ID único da transação	sim	N50	Devolvido pela função “getResult”
payment.amount	Valor da transação	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	sim	N3	200 Total (Full) 201 Parcial (Partial)
reference	Referência do comerciante	não	AN50	
comment	Comentário / observação	não	AN255	

Exemplo de um pedido em formato REST:

```
{
  "method": "doCapture",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "transaction": {
    "id": "R44wEAdFbR4z7ZZ5A0AJ"
  },
  "payment": {
    "amount": 700,
    "action": 200
  }
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado:	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
transaction.id	ID único da transação	AN50	
transaction.date	Data e hora da transação	AN16	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Sucesso 2 – Erro

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "90000000000",
    "message": "Success"
  },
  "transaction": {
    "id": "QfgUFEPRszj0zTX6SpX5",
    "date": "2023-04-04 11:56:10",
    "status": "1"
  }
}
```

A função “doRefund” permite ao comerciante reembolsar um pagamento autorizado e confirmado.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	doRefund
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
transaction.id	ID único da transação	sim	N50	Devolvido pela função “getResult”
payment.amount	Valor da transação	sim	N12	
payment.action	Código da ação de pagamento	sim	N4	300 Reembolso (Refund)
reference	Referência do comerciante	não	AN50	
comment	Comentário / observação	não	AN255	

Exemplo de um pedido em formato REST:

```
{
  "method": "doRefund",
  "api" : {
    "username": "demouser",
    "password": "abc123"
  },
  "transaction": {
    "id": "QfgUFEPRszj0zTX6SpX5"
  },
  "payment": {
    "amount": 700,
    "action": 300
  },
  "comment": "devolvido"
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado:	AN11	Ver em anexo “Códigos e mensagens”
result.message	Mensagem do resultado	AN255	
transaction.id	ID único da transação	AN50	
transaction.date	Data e hora da transação	AN16	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Sucesso 2 – Erro

Exemplo de uma resposta em formato REST:

```
{
```

```

"result": {
  "code": "9000000000",
  "message": "Success"
},
"transaction": {
  "id": "cXDP8hTmYkdzX5NCsHu8",
  "date": "2023-04-04 11:57:29",
  "status": "1"
}
}

```

doVoid

A função “doVoid”, dependendo do meio de pagamento utilizado, permite cancelar um pagamento autorizado.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	doVoid
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
transaction.id	ID único da transação	sim	N50	Devolvido pela última operação
reference	Referência do comerciante	não	AN50	
comment	Comentário / observação	não	AN255	

Exemplo de um pedido em formato REST:

```

{
  "method": "doVoid",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "transaction": {
    "id": "14094160626454"
  }
}

```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado:	AN11	Ver em anexo “Códigos e mensagens”
result.message	Mensagem do resultado	AN255	
transaction.id	ID único da transação	AN50	
transaction.date	Data e hora da transação	AN16	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Sucesso

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "7000000000",
    "message": "Request successfully processed in 'Merchant in Connector Test Mode'"
  },
  "transaction": {
    "id": "8ac7a4a1874921bd018751ab5d5f06a4",
    "date": "2023-04-05 14:46:38",
    "status": "1"
  }
}
```

searchTransactions

A função “searchTransactions” permite pesquisar transações.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	searchTransactions
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
filter.startDate	Transações iniciadas a partir de	não	AN10	2025-12-01
filter.endDate	Transações iniciadas até	não	AN10	2025-12-31
filter.orderRef	Referência do pagamento	não	AN50	
filter.transactionId	ID da transação	não	AN50	
filter.solution	Solução de pagamento	não	N3	100 - REDUNIQ E-Commerce 101 - PayPal (SOAP) 102 - AMEX (3C iPage) ...
filter.status	Estado da transação	não	N1	1 – Aguarda seleção do meio de pagamento 2 – Em curso 3 – Erro / terminada 4 – Sucesso / terminada
offset	Define a primeira posição a ser devolvida	não	N	
limit	Limite de registos a devolver	não	N4	1 a 2000 registos

Exemplo de um pedido em formato REST:

```
{
  "method": "searchTransactions",
  "api" : {
    "username": "demouser",
    "password": "abc123"
  },
  "filter" : {
    "startDate": "2025-12-01",
    "endDate": "2025-12-31",
    "orderRef": "",
    "transactionId": "7649524475306028204806",
    "solution": "",
    "status": "4"
  },
  "offset": 0,
  "limit": 10
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado:	AN11	Ver em anexo “Códigos e mensagens”
result.message	Mensagem do resultado	AN255	
transactions	Lista de transações		
offset	Primeira posição a ser devolvida	N	
limit	Nº de registos devolvidos	N	
totalCount	Total de registos que correspondem ao filtro	N	

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "transactions": [
    {
      "result": {
        "code": "14000000000",
        "message": "Success"
      },
      "transaction": {
        "id": "7649524475306028204806",
        "isFraud": "0",
        "date": "2025-12-05 16:34:07+00",
        "status": "4"
      }
    }
  ]
}
```

```

    },
    "payment": {
      "amount": "100",
      "action": "100",
      "solution": "114",
      "deadline": null
    }
  },
  "offset": 0,
  "limit": 1,
  "totalCount": 1
}

```

createPaymentToken

A função “createPaymentToken” permite criar *tokens* de pagamento que podem ser utilizados em pagamentos recorrentes ou ocasionais.

Estrutura de dados do pedido:

Campo	Descrição	Obrig.	Formato	Observações
method	Nome da função	sim	AN50	createPaymentToken
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payment.solution	Código da solução de pagamento a utilizar.	sim	N3	113 – Cartão de crédito (CyberSource) 117 – Click to Pay (CyberSource)
payment.description	Descrição do pagamento	sim	AN255	
buyer.firstName	Nome do cliente	sim	AN50	
buyer.lastName	Sobrenome do cliente	sim	AN50	
buyer.email	E-mail do cliente	sim	AN150	
buyer.phone	Telefone do cliente	não	AN15	Exemplos: 910000000 00351910000000 +351910000000
buyer.billing.street1	Faturação: Rua 1	sim	AN100	
buyer.billing.street2	Faturação: Rua 2	não	AN100	
buyer.billing.city	Faturação: Cidade	sim	AN40	
buyer.billing.state	Província/estado	não	AN40	
buyer.billing.zipCode	Faturação: Código postal	não	AN20	
buyer.billing.country	Faturação: País	sim	AN2	ISO 3166-1 (exemplo: “pt”)
buyer.billing.tin	Número de identificação fiscal	não	AN20	
payToken.action	Código para criação de um <i>token</i> de pagamento	sim	N3	500 – Criar <i>token</i> de pagamento
payToken.shared	Partilha do token	não	N1	0 – token não partilhado 1 – token partilhado
payToken.ref	Referência do token	sim	AN50	
card.number	Número do cartão	sim	N20	

card.expMonth	Mês de expiração do cartão	sim	N2	
card.expYear	Ano de expiração do cartão	sim	N4	
card.secCode	Código de segurança do cartão	não	N4	

Exemplo de um pedido em formato REST:

```
{
  "method": "createPaymentToken",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "payment": {
    "solution": "113",
    "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
  },
  "buyer": {
    "firstName": "Manuel",
    "lastName": "Silva",
    "email": "manuel.silva@mail.com",
    "phone": "",
    "billing": {
      "street1": "Rua Alberto Sampaio",
      "street2": "n50, 2D",
      "city": "Lisboa",
      "state": "Lisboa",
      "zipCode": "2100-456",
      "country": "pt"
    }
  },
  "payToken": {
    "action": 500,
    "shared": 1,
    "ref": "ref1716454408"
  },
  "card": {
    "number": "4111111111111111",
    "expMonth": "12",
    "expYear": "2031",
    "secCode": "123"
  }
}
```

Estrutura de dados da Resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 130000000000 - OK	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
payToken.id	ID do <i>token</i>	N20	

payToken.sharedKey	Chave partilhada do <i>token</i> de pagamento	AN36	
payToken.status	Estado do <i>token</i> de pagamento	N1	1 – Sucesso / autorizado 2 – Erro / não autorizado

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "130000000000",
    "message": "AUTHORIZED"
  },
  "payToken": {
    "id": "100009",
    "sharedKey": "2971a8f8-68d4-41d9-8452-12e03b57efcc",
    "status": "1"
  }
}
```

disablePaymentToken

A função “disablePaymentToken” permite ao comerciante desativar um *token* de pagamento.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	disablePaymentToken
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payToken.id	ID do <i>token</i> de pagamento	sim	N8	
payToken.action	Código para desativar de um <i>token</i> de pagamento	sim	N3	502 – Desativar <i>token</i> de pagamento

Exemplo de um pedido em formato REST:

```
{
  "method": "disablePaymentToken",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "payToken": {
    "id": "115",
    "action": "502"
  }
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 00000000 - OK	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
payToken.id	ID do <i>token</i> de pagamento	N8	
payToken.status	Estado do <i>token</i> de pagamento	N1	1 – Sucesso / autorizado 2 – Erro / não autorizado 3 - Inativo

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "payToken": {
    "id": "115",
    "status": "3"
  }
}
```

createRecurringPayment

A função "*createRecurringPayment*" permite criar pagamentos recorrentes utilizando um *token* de pagamento previamente criado.

Estrutura de dados do pedido:

Campo	Descrição	Obrig.	Formato	Observações
method	Nome da função	sim	AN50	createRecurringPayment
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payment.amount	Valor total a pagar	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	sim	N3	100 – Autorização 101 – Autorização + Captura
payment.description	Descrição do pagamento	sim	AN255	
order.ref	Referência do pagamento	sim	AN50	
recurring.action	Código para criação de um pagamento recorrente	sim	N3	600 – Criar pagamento recorrente
recurring.startDate	Data de início	sim	AN10	2024-05-23
recurring.intervalUnit	Tipo de intervalo (dia, mês ou ano)	sim	AN5	Valores possíveis: - day - month - year

recurring.intervalCount	Número de recorrências	sim	N4	
payToken.id	ID do <i>token</i> de pagamento	não ⁽¹⁾	N8	ID devolvido pela função "createPaymentToken" ou "getResult" Nota: este campo só pode ser usado pelo "dono" do <i>token</i>
payToken.sharedKey	Chave partilhada do <i>token</i> de pagamento	não ⁽¹⁾	AN36	ID devolvido pela função "createPaymentToken" ou "getResult"
notificationUrl	URL de notificação	não	AN255	
languageCode	Código do idioma	não	AN6	ISO 639-2 (exemplo: "por")

(1) Indicar um dos campos.

Exemplo de um pedido em formato REST:

```
{
  "method": "createRecurringPayment",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "payment": {
    "amount": "700",
    "action": "100",
    "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
  },
  "order": {
    "ref": "ref1716454408"
  },
  "recurring": {
    "action": "600",
    "startDate": "2024-05-23",
    "intervalUnit": "day",
    "intervalCount": "7"
  },
  "payToken": {
    "id": "100009"
  },
  "notificationUrl": "https://www.merchant.pt/notification/",
  "languageCode": "por"
}
```

Estrutura de dados da Resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 00000000 - OK	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
recurring.id	ID do pagamento recorrente	N20	

recurring.startDate	Data de início	N10	
recurring.startEnd	Date de fim	N10	
recurring.intervalUnit	Tipo de intervalo (dia, mês ou ano)	N5	
recurring.intervalCount	Número de recorrências	N4	
recurring.status	Estado do pagamento recorrente	N1	1 – Sucesso / ativo 2 – Erro / inativo

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "recurring": {
    "id": "100004",
    "status": "1"
  }
}
```

getRecurringPayment

A função “getRecurringPayment” permite ao comerciante consultar os detalhes de um pagamento recorrente.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	getRecurringPayment
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
recurring.id	ID do pagamento recorrente	sim	N8	
recurring.action	Código para consulta do detalhe de um pagamento recorrente	sim	N3	601 – Detalhe do pagamento recorrente

Exemplo de um pedido em formato REST:

```
{
  "method": "getRecurringPayment",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "recurring": {
    "id": "100004",
    "action": "601"
  }
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 00000000 - OK	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
payment.amount	Valor total do pagamento	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	N4	100 – Autorização 101 – Autorização + Captura
payment.description	Descrição do pagamento	AN255	
order.ref	Referência do pagamento	AN50	
buyer.firstName	Nome do cliente	AN50	
buyer.lastName	Sobrenome do cliente	AN50	
buyer.email	E-mail do cliente	AN150	
buyer.phone	Telefone do cliente	AN15	
buyer.billing.street1	Expedição: Rua 1	AN100	
buyer.billing.street2	Expedição: Rua 2	AN100	
buyer.billing.city	Expedição: Cidade	AN40	
buyer.billing.state	Província/estado	AN40	
buyer.billing.zipCode	Expedição: Código postal	AN20	
buyer.billing.country	Expedição: País	AN2	ISO 3166-1 (exemplo: "pt")
recurring.id	ID do pagamento recorrente	N8	
recurring.startDate	Data de início	AN10	
recurring.endDate	Data de fim	AN10	
recurring.intervalUnit	Tipo de intervalo (dia, mês ou ano)	AN5	Valores possíveis: - day - month - year
recurring.intervalCount	Número de recorrências	N4	
recurring.status	Estado do pagamento recorrente	N1	1 – Sucesso / ativo 2 – Erro / inativo 3 - Concluído
payToken.id	ID do <i>token</i> de pagamento	N8	
payToken.status	Estado do <i>token</i> de pagamento	N1	1 – Sucesso / autorizado 2 – Erro / não autorizado 3 - Inativo
notificationUrl	URL de notificação	AN255	
languageCode	Código do idioma	AN6	ISO 639-2 (exemplo: "por")

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "payment": {
```



```

    "amount": "700",
    "action": "100",
    "description": "Lorem ipsum dolor sit amet, consectetur"
  },
  "order": {
    "ref": "ref1716454408"
  },
  "order": {
    "firstName": "Manuel",
    "lastName": "Silva",
    "email": "manuel.silva@mail.com",
    "phone": "",
    "billing": {
      "street1": "Av. António Augusto Aguiar, 122",
      "street2": "",
      "city": "Lisboa",
      "state": "Lisboa",
      "zipCode": "1050-019",
      "country": "PT"
    }
  },
  "recurring": {
    "id": "100004",
    "startDate": "2024-05-23",
    "endDate": "2024-05-29",
    "intervalUnit": "day",
    "intervalCount": "7",
    "status": "1"
  },
  "payToken": {
    "id": "100009",
    "status": "1"
  },
  "notificationUrl": "https://www.merchant.pt/notification/",
  "languageCode": "por"
}

```

disableRecurringPayment

A função “disableRecurringPayment” permite ao comerciante desativar um pagamento recorrente.

Estrutura de dados do pedido:

Campo	Descrição	Obrigatório	Formato	Observações
method	Nome da função	sim	AN50	disableRecurringPayment
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
recurring.id	ID do pagamento recorrente	sim	N8	
recurring.action	Código para desativar de um pagamento recorrente	sim	N3	602 – Desativar pagamento recorrente

Exemplo de um pedido em formato REST:

```
{
  "method": "disableRecurringPayment",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "recurring": {
    "id": "100004",
    "action": "602"
  }
}
```

Estrutura de dados da resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 00000000 - OK	AN11	Ver em anexo "Códigos e mensagens"
result.message	Mensagem do resultado	AN255	
recurring.id	ID do pagamento recorrente	N8	
recurring.status	Estado do pagamento recorrente	N1	1 – Sucesso / ativo 2 – Erro / inativo 3 - Concluído
payToken.id	ID do <i>token</i> de pagamento	N8	
payToken.status	Estado do <i>token</i> de pagamento	N1	1 – Sucesso / autorizado 2 – Erro / não autorizado 3 - Inativo

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "recurring": {
    "id": "100004",
    "status": "3"
  },
  "payToken": {
    "id": "100009",
    "status": "1"
  }
}
```

A função “doPaymentToken” permite ao comerciante realizar um pagamento utilizando um *token* de pagamento previamente criado.

Estrutura de dados do pedido:

Campo	Descrição	Obrig.	Formato	Observações
method	Nome da função	sim	AN50	doPaymentToken
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payment.amount	Valor total a pagar (order amount + order taxes + shipping amount)	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
payment.action	Código da ação de pagamento	sim	N3	100 – Autorização 101 – Autorização + Captura
payment.description	Descrição do pagamento	sim	AN255	
order.ref	Referência do pagamento	sim	AN50	Cada pagamento deve ter uma referência única
order.amount	Valor a pagar (sem impostos e sem transporte)	sim	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
order.taxes	Valor total dos impostos	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€
order.date	Data e hora do pagamento	sim	AN19	2012-12-15 20:30:00
order.shipping	Expedição / entrega	não	N1	0 – Não (por omissão) 1 – Sim Se existe expedição e o order.details não é indicado, é considerado um bem “Físico/Material”
order.details	Detalhes do pagamento	não		Ver estrutura de dados “order.details”
buyer.shipping.name	Nome da pessoa associada a este endereço	não	AN32	
buyer.shipping.street1	Expedição: Rua 1	não	AN100	
buyer.shipping.street2	Expedição: Rua 2	não	AN100	
buyer.shipping.city	Expedição: Cidade	não	AN40	
buyer.shipping.state	Província/estado	não	AN40	
buyer.shipping.zipCode	Expedição: Código postal	não	AN20	
buyer.shipping.country	Expedição: País	não	AN2	ISO 3166-1 (exemplo: “pt”)
buyer.shipping.phone	Expedição: Telefone	não	AN15	
buyer.shipping.amount	Expedição: Valor	não	N12	O valor 100 corresponde a 1€ O valor 1599 corresponde a 15,99€ Quando definido, o order.shipping deve ser igual a “1”

payToken.id	ID do <i>token</i> de pagamento	não ⁽¹⁾	N8	ID devolvido pela função "createPaymentToken" ou "getResult" Nota: este campo só pode ser usado pelo "dono" do <i>token</i>
payToken.sharedKey	Chave partilhada do <i>token</i> de pagamento	não ⁽¹⁾	AN36	ID devolvido pela função "createPaymentToken" ou "getResult"
payToken.action	Código para efetuar um pagamento	sim	N3	503 – Efetuar pagamento
privateData	Dados privados	não		Ver estrutura de dados "privateData"
languageCode	Código do idioma	não	AN6	ISO 639-2 (exemplo: "por")

(1) Indicar um dos campos.

Exemplo de um pedido em formato REST:

```
{
  "method": "doPaymentToken",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "payment": {
    "amount": 700,
    "action": 100,
    "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
  },
  "order": {
    "ref": "ref1680534950",
    "amount": 500,
    "taxes": 100,
    "date": "2023-04-03 16:15:42",
    "shipping": "1",
    "details": [
      {
        "name": "Item #1",
        "amount": 250,
        "tax": 50,
        "quantity": 1,
        "category": "1"
      },
      {
        "name": "Item #2",
        "amount": 250,
        "tax": 50,
        "quantity": 1,
        "category": "1"
      }
    ]
  },
  "buyer": {
    "shipping": {
```

```

    "name": "Manuel Silva",
    "street1": " Rua Alberto Sampaio",
    "street2": "n50, 2D",
    "city": " Lisboa",
    "state": " Lisboa",
    "zipCode": "2100-456",
    "country": "pt",
    "phone": "+351213456784",
    "amount": 100
  }
},
"payToken": {
  "id": "100009",
  "action": 503
},
"privateData": [
  {
    "name": "key #1",
    "value": "value #1"
  },
  {
    "name": "key #2",
    "value": "value #2"
  },
  {
    "name": "key #3",
    "value": "value #3"
  }
],
"languageCode": "por"
}

```

Estrutura de dados da Resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 13000000000 - OK	AN11	Ver em anexo “Códigos e mensagens”
result.message	Mensagem do resultado	AN255	
transaction.id	ID único da transação	AN50	
transaction.isFraud		AN1	Disponível para o Reduniq E-commerce 0 – risco de fraude não detectado 1 – risco de fraude
transaction.date	Data e hora da transação	AN19	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Aguarda seleção do meio de pagamento 2 – Em curso 3 – Erro / terminada 4 – Sucesso / terminada
transaction.extraData	Informação adicional associada à transação		Ver estrutura de dados “transaction.extraData”
token	token atribuído ao pagamento	AN50	

Exemplo de uma resposta em formato REST:

```
{
  "result": {
    "code": "13000000000",
    "message": "AUTHORIZED"
  },
  "transaction": {
    "id": "7164633657866157404951",
    "isFraud": "0",
    "extraData": [
      {
        "name": "",
        "value": ""
      }
    ],
    "status": "4"
  },
  "token": "70cd8dd29666fd1d335b0111c9054071"
}
```

doPaymentCard

A função “doPaymentCard” permite ao comerciante realizar um pagamento utilizando um cartão. A utilização desta função requer uma chamada prévia à função “initPayment” (*mode=webless*).

Estrutura de dados do pedido:

Campo	Descrição	Obrig.	Formato	Observações
method	Nome da função	sim	AN50	doPaymentCard
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
token	token atribuído ao pagamento	sim	AN50	Token devolvido pela função “initPayment” (<i>mode=webless</i>)
card.number	Número do cartão	sim	N20	
card.expMonth	Mês de expiração do cartão	sim	N2	
card.expYear	Ano de expiração do cartão	sim	N4	
card.secCode	Código de segurança do cartão	não	N4	
authentication.step	Fase do processo (autenticação 3DS)	sim	N1	1 - Device Data Collection 2 - PA Check Enrollment Service 3 - PA Validation Service (only for step-up authentication)
authentication.referenceId	ID da sessão de autenticação	sim ⁽¹⁾	AN36	Valor obtido no step 1

authentication.acsWindowSize	Tamanho da janela de desafio (página do emissor)	sim ⁽¹⁾	N2	01 - 250x400 02 - 390x400 03 - 500x600 04 - 600x400 05 - Full page
authentication.returnUrl	URL de retorno do comerciante	sim ⁽²⁾	AN255	URL onde é feito um pedido POST (pelo emissor) com o <i>transactionId</i> da autenticação
authentication.transactionId	ID da transação de autenticação	sim ⁽²⁾	AN20	Valor obtido pela página de retorno do comerciante

(1) obrigatório para o step 2

(2) obrigatório para o step 3

Exemplo de um pedido em formato REST (step 1 - Device Data Collection):

```
{
  "method": "doPaymentCard",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "token": "4f5c8617a09c1bf7b52e149ffe5a5d6908fc5130",
  "card": {
    "number": "4456530000001096",
    "expMonth": "12",
    "expYear": "2031",
    "secCode": "123"
  },
  "authentication": {
    "step": 1
  }
}
```

Com os dados obtidos na resposta (step 1 – ver exemplo da resposta), o sistema do comerciante deve iniciar a recolha dos dados do dispositivo no *browser* do cliente (*client side*). Ver “Exemplo simplificado de uma página HTML com o *workflow* da autenticação 3DS”.

Exemplo de um pedido em formato REST (step 2 - PA Check Enrollment Service):

```
{
  "method": "doPaymentCard",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "token": "4f5c8617a09c1bf7b52e149ffe5a5d6908fc5130",
  "card": {
    "number": "4456530000001096",
```

```

    "expMonth": "12",
    "expYear": "2031",
    "secCode": "123"
  },
  "authentication": {
    "step": 2,
    "referenceId": "cae1881a-ca2e-4165-8a09-a47260ad0247",
    "acsWindowSize": "05",
    "returnUrl": " https://www.merchant.pt/return3dschallenge"
  }
}

```

Com os dados obtidos na resposta (step 2 – ver exemplo da resposta), caso seja solicitado um *step up*, o sistema do comerciante deve iniciar a página de desafio do emissor no *browser* do cliente (*client side*). Ver “Exemplo simplificado de uma página HTML com o workflow da autenticação 3DS”.

Exemplo de um pedido em formato REST (step 3 - PA Validation Service):

```

{
  "method": "doPaymentCard",
  "api": {
    "username": "demouser",
    "password": "abc123"
  },
  "token": "4f5c8617a09c1bf7b52e149ffe5a5d6908fc5130",
  "card": {
    "number": "4456530000001096",
    "expMonth": "12",
    "expYear": "2031",
    "secCode": "123"
  },
  "authentication": {
    "step": 3,
    "transactionId": "E6VZzpuilt2pFwsbCML0"
  }
}

```

Estrutura de dados da Resposta:

Campo	Descrição	Formato	Observações
result.code	Código do resultado: 170000000000 - OK	AN11	Ver em anexo “Códigos e mensagens”
result.message	Mensagem do resultado	AN255	
authentication.accessToken	Token de sessão	JWT	Valor devolvido no step 1 e step 2 (em caso de <i>step up</i>)
authentication.deviceDataCollectionUrl	URL da página para recolha dos dados do dispositivo	AN255	Valor devolvido no step 1

authentication.referenceId	ID da sessão de autenticação	AN36	Valor devolvido no step 1
authentication.stepUpUrl	URL da página de desafio	AN255	Valor devolvido no step 2 (em caso de <i>step up</i>)
authentication.nextStep	Indicação do próximo passo	N1	
transaction.id	ID único da transação	AN50	
transaction.isFraud		AN1	Disponível para o Reduniq E-commerce 0 – risco de fraude não detectado 1 – risco de fraude
transaction.date	Data e hora da transação	AN19	2012-12-15 20:30:00
transaction.status	Estado da transação	N1	1 – Aguarda seleção do meio de pagamento 2 – Em curso 3 – Erro / terminada 4 – Sucesso / terminada
transaction.extraData	Informação adicional associada à transação		Ver estrutura de dados "transaction.extraData"

Exemplo de uma resposta em formato REST (step 1):

```
{
  "result": {
    "code": "00800001",
    "message": "Waiting for confirmation of a pending transaction"
  },
  "authentication": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
    "deviceDataCollectionUrl": "https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect",
    "referenceId": "58438126-9a9e-4aba-859c-98c2350b0bc4",
    "nextStep": 2
  },
  "transaction": {
    "id": null,
    "isFraud": "0",
    "date": "2025-09-25 15:24:40+01",
    "status": "2",
    "extraData": []
  }
}
```

Exemplo de uma resposta em formato REST (step 2 – com pedido de *step up*):

```
{
  "result": {
    "code": "00800001",
    "message": "Waiting for confirmation of a pending transaction"
  },
  "authentication": {
```

```

    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI0MGEzMD...",
    "stepUpUrl": "https://centinelapistag.cardinalcommerce.com/V2/Cruise/StepUp",
    "nextStep": 3
  },
  "transaction": {
    "id": null,
    "isFraud": "0",
    "date": "2025-09-25 18:01:07+01",
    "status": "2",
    "extraData": []
  }
}

```

Exemplo de uma resposta em formato REST (step 3):

```

{
  "result": {
    "code": "17000000000",
    "message": "Success"
  },
  "authentication": [],
  "transaction": {
    "id": "7588200182406149104806",
    "isFraud": "0",
    "date": "2025-09-25 18:06:59+01",
    "status": "4",
    "extraData": []
  }
}

```

Exemplo simplificado de uma página HTML com o *workflow* da autenticação 3DS

```

<!DOCTYPE html>
<html>
<head>
  <title>Payment Form</title>
</head>

<body>
  <h1>Payment Form</h1>

  <form action="/submit-payment" method="post" id="formPaymentID" name="formPayment">
    <input name="referenceId" type="hidden" value="" />
    <input name="transactionId" type="hidden" value="" />
    <input name="step" type="hidden" value="1" />

    card.number <input name="card_number" size="20" value="4456530000001096" /> <br>
    card.expMonth <input name="card_expMonth" size="2" value="12" /> <br>
    card.expYear <input name="card_expYear" size="4" value="2031" /> <br>
    card.secCode <input name="card_secCode" size="4" value="123" /> <br>
  </form>

```

```

<input type="reset" value="Reset" />
<input value="Submit" type="submit" />
</form>

<iframe id="cardinalCollectionIframeID" name="collectionIframe" height="1" width="1"
style="display:none;"></iframe>
<form id="cardinalCollectionFormID" method="POST" target="collectionIframe" action="">
  <input type="hidden" name="JWT" value="">
</form>

<iframe id="stepUpIframeID" name="stepUpIframe" width="100%" height="600" frameborder="0"
scrolling="auto"></iframe>
<form id="stepUpFormID" method="POST" target="stepUpIframe" action="">
  <input type="hidden" name="JWT" value="">
  <input type="hidden" name="MD" value="">
</form>
</body>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

<script>
$(function() {
  $("#formPaymentID").on("submit", function(e) {
    e.preventDefault();

    // step 1 - Device Data Collection
    $.ajax({
      url: $(this).prop('action'),
      type: "POST",
      data: $(this).serialize(),
      dataType: "json",
      success: function (d) {
        $('#formPaymentID input[name="referenceId"]').val(d.authentication.referenceId);
        // Init Device Data Collection
        $('#cardinalCollectionFormID input[name="JWT"]').val(d.authentication.accessToken);
        $('#cardinalCollectionFormID').attr('action', d.authentication.deviceDataCollectionUrl);
        $('#cardinalCollectionFormID').submit();
      }
    });
  });
});

window.addEventListener('message', function(event) {
  // message from cardinal page (device data collection)
  if (
    event.origin === 'https://centinelapistag.cardinalcommerce.com' ||
    event.origin === 'https://centinelapi.cardinalcommerce.com'
  ) {
    var data = JSON.parse(event.data);
    var referenceId = $('#formPaymentID input[name="referenceId"]').val();
    if (
      data.SessionId == referenceId
    ) {
      // step 2 - PA Check Enrollment Service

```

```

$('#formPaymentID input[name="step"]').val('2');

$.ajax({
  url: $('#formPaymentID').prop('action'),
  type: "POST",
  data: $('#formPaymentID').serialize(),
  dataType: "json",
  success: function (d) {
    // Init PA Validation Service (only for step-up authentication)
    $('#stepUpFormID input[name="JWT"]').val(d.authentication.accessToken);
    $('#stepUpFormID').attr('action', d.authentication.stepUpUrl);
    $('#stepUpFormID').submit();
  }
});
}
}

var originUrl = window.location.protocol + "://" + window.location.host;

// message from merchant return page (3ds step-up challenge page)
if (
  event.origin === originUrl
) {
  var data = JSON.parse(event.data);
  if (
    data.TransactionId
  ) {
    // step 3 - PA Validation Service
    $('#formPaymentID input[name="step"]').val('3');
    $('#formPaymentID input[name="transactionId"]').val(data.TransactionId);
    $('#formPaymentID').submit();
  }
}, false);
</script>
</html>

```

Exemplo simplificado, em PHP, da página de retorno da autenticação

```

<?php
// init
$TransactionId = (isset($_POST['TransactionId'])) ? $_POST['TransactionId'] : "";
?>
<!DOCTYPE html>
<html>
<head>
<title>3DS Return Page</title>
</head>

<body>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

```

```
<script>
$(function () {
  var message = JSON.stringify({
    TransactionId: '<?php echo $TransactionId; ?>'
  });
  var originUrl = window.location.protocol + "://" + window.location.host;
  window.parent.postMessage(message, originUrl);
});
</script>
</body>
</html>
```

A função “doPayByLink” gera um novo pedido de pagamento.

Parâmetros (formato JSON):

Campo	Descrição	Obrig.	Tipo	Observações
method	Nome da função	sim	AN50	doPayByLink
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
payByLink.firstName	Nome	(1)	AN150	
payByLink.lastName	Apelido	(1)	AN150	
payByLink.email	Email	(1)	AN150	
payByLink.phone	Telefone	(1)	AN20	
payByLink.addressLine1	Morada (linha 1)	(1)	AN250	
payByLink.addressLine2	Morada (linha 2)	(1)	AN250	
payByLink.zipCode	Código postal	(1)	AN50	
payByLink.locality	Localidade	(1)	AN100	
payByLink.country	País	(1)	AN2	ISO 3166-1 (exemplo: “pt”)
payByLink.description	Descrição	(1)	AN150	
payByLink.referenceEntity	Referência da entidade	(1)	N5	Quando o campo “Referência MB com entidade” está ativo no formulário.
payByLink.referenceNumber	Referência	(1)	AN50	Referência alfanumérica ou referência MB (depende da configuração do formulário)
payByLink.taxpayerNumber	Nº de contribuinte	(1)	AN50	
payByLink.paymentDeadline	Data limite de pagamento	sim	Date	dd-mm-aaaa
payByLink.numDaysAsyncPay	Nº de dias, a contar da data do pagamento, de validade da referência. (pagamento assíncrono)	não	N2	0-31
payByLink.amount	Valor a pagar	sim	N8,2	5.00 (cinco euros) 10.50 (dez euros e cinquenta cêntimos)
payByLink.paymentSolution	Solução de pagamento	não	N3	100 - REDUNIQ E-Commerce 101 - PayPal (SOAP) 102 - 3C iPage 105 - PayPal (REST) 106 - Cartão de crédito (DPG) 107 - MB WAY (DPG) 108 - Pagamento de Serviços (DPG) 109 - Cartão de crédito (SPG) 110 - MB WAY (SPG) 111 - Pagamento de Serviços (SPG) 112 - Parcela Já

				113 – Cartão de crédito (Cybersource) 114 - Google Pay (Cybersource) 115 - Apple Pay (Cybersource) 116 - PIX (Braza)
payByLink.language	Idioma	não	S2	pt – português en – inglês es – espanhol fr – francês de – alemão
payByLink.sendLinkByEmail	Enviar URL do formulário por email	não	N1	1 – sim 2 – não (omissão)
payByLink.emailWithQrCode	Enviar código QR no email	não	N1	1 – sim (omissão) 2 – não
payByLink.sendLinkBySMS	Enviar URL do formulário por SMS	não	N1	1 – sim 2 – não (omissão)
payByLink.mobileNumSMS	Nº de telemóvel	não	S20	O indicativo do país é opcional. Exemplos: 00351900000000 +351900000000 900000000
payByLink.sendLinkByWhatsApp	Enviar URL do formulário por WhatsApp	não	N1	1 – sim 2 – não (omissão)
payByLink.mobileNumWhatsApp	Nº do WhatsApp	não	S20	O indicativo do país é opcional. Exemplos: 00351900000000 +351900000000 900000000
payByLink.showPaymentForm	Mostrar formulário de pagamento	não	N1	1 – sim (omissão) 2 – não
payByLink.canChangeFormData	Pode alterar os dados do formulário	não	N1	1 – sim 2 – não (omissão)
payByLink.recurringPayType	Tipo de pagamento recorrente	não	N1	1 – Valor variável 2 – Valor fixo
payByLink.recurringPayStart	Início (dias)	(2)	N2	O primeiro pagamento recorrente será efetuado X dias após a data de pagamento desta solicitação de pagamento.
payByLink.recurringPayAmount	Valor	(2)	N8,2	5.00 (cinco euros) 10.50 (dez euros e cinquenta cêntimos)
payByLink.recurringPayUnit	Frequência (unidade)	(2)	S5	Valores possíveis: day month year
payByLink.recurringPayCount	Nº ocorrências	(2)	N4	
payByLink.notificationURL	URL de notificação	não	AN255	Recebe a resposta assíncrona do resultado do pagamento
payByLink.targetOriginUrl	URL de origem (loja do comerciante)	não	AN255	Importante quando os <i>widgets lightbox</i> ou <i>iframe</i> são utilizados
payByLink.returnUriOk	URL de retorno para a página do comerciante	não	AN255	Usado quando o pagamento é aceite.

paymentData.returnUrlError	URL de retorno para a página do comerciante	não	AN255	Usado quando o pagamento é recusado ou cancelado.
----------------------------	---	-----	-------	---

- (1) A obrigatoriedade destes campos depende da configuração do formulário no *Backoffice* do comerciante.
- (2) Campos obrigatórios para pagamentos recorrentes de “valor fixo”.

Exemplo (POST em formato JSON):

```
{
  "method": "doPayByLink",
  "api": {
    "username": "999999999",
    "password": "e6c5a00baecf"
  },
  "payByLink": {
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@fakemail.com",
    "phone": "910000000",
    "addressLine1": "Rua da Liberdade",
    "addressLine2": "315",
    "zipCode": "2000-100",
    "locality": "Lisboa",
    "country": "pt",
    "description": "Hotel reserve",
    "referenceNumber": "ref1728596129",
    "taxpayerNumber": "204884884",
    "paymentDeadline": "20-10-2025",
    "numDaysAsyncPay": 3,
    "amount": "100.00",
    "language": "pt",
    "sendLinkByEmail": "1",
    "recurringPayType": "",
    "recurringPayStart": "",
    "recurringPayAmount": "",
    "recurringPayUnit": "",
    "recurringPayCount": "",
    "notificationURL": "http://www.merchant-domain.com/",
    "returnUrlOk": "",
    "returnUrlError": ""
  }
}
```

Nota: O endereço *notificationUrl* será invocado com o resultado do pedido de pagamento. A resposta é submetida em formato JSON, conforme exemplo:

```
{
  "payByLink": {
    "status": "2"
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```


Resposta da função “doPayByLink” (formato JSON):

```
{
  "result": {
    "code": "000000000",
    "message": ""
  },
  "payByLink": {
    "url": "https://pagamentos.sandbox.reduniq.pt/pay-by-link/339933/...",
    "qrCode": {
      "mimetype": "image/png",
      "base64": "iVBORw0KGgoAAAANSUhEUgAAAUAFAAACXB..."
    },
    "status": "1"
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```

resultPayByLink

A função “resultPayByLink” permite obter o estado de um determinado pedido de pagamento.

Parâmetros (formato JSON):

Campo	Descrição	Obrig.	Tipo	Observações
method	Nome da função	sim	AN50	resultPayByLink
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
token	Token do pedido de pagamento	sim	AN50	

Exemplo (POST em formato JSON):

```
{
  "method": "resultPayByLink",
  "api": {
    "username": "999999999",
    "password": "e6c5a00baecf"
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```

Resposta da função “resultPayByLink” (formato JSON):

```
{
  "result": {
    "code": "000000000",
    "message": ""
  },
  "payByLink": {
```

```

    "url": "https://pagamentos.sandbox.reduniq.pt/pay-by-link/339933/...",
    "qrCode": {
      "mimetype": "image/png",
      "base64": "iVBORw0KGgoAAAANSUhEUgAAAUAAL9AAAACX..."
    },
    "status": "5"
  },
  "transaction": {
    "id": "7653817998846582604805",
    "date": "2025-12-10 15:50:01+00",
    "status": "4",
    "extraData": []
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}

```

Valores possíveis de “**status**”:

- 1 = Em pagamento
- 2 = Terminado (autorizado)
- 3 = Terminado (não autorizado)
- 4 = Expirado
- 5 = Cancelado
- 6 = Em curso
- 7 = Aguarda confirmação

voidPayByLink

A função “*voidPayByLink*” permite cancelar um pedido de pagamento no estado “Em pagamento”.

Parâmetros (formato JSON):

Campo	Descrição	Obrig.	Tipo	Observações
method	Nome da função	sim	AN50	voidPayByLink
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
token	Token do pedido de pagamento	sim	AN50	

Exemplo (POST em formato JSON):

```

{
  "method": "voidPayByLink",
  "api": {
    "username": "999999999",
    "password": "e6c5a00baecf"
  },
}

```

```
{
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```

Resposta da função “voidPayByLink” em caso de sucesso (formato JSON):

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "payByLink": {
    "status": "5"
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```

Resposta da função “voidPayByLink” em caso de erro (formato JSON):

```
{
  "result": {
    "code": "00900106",
    "message": "This payment request cannot be canceled."
  },
  "payByLink": {
    "status": null
  },
  "token": "d332ddabb7622acf6b92059bef9cf55498d9c9f7"
}
```

regenPayByLink

A função “regenPayByLink” permite renovar um pedido de pagamento no estado “Terminado (não autorizado)”, “Expirado” ou “Cancelado”.

Parâmetros (formato JSON):

Campo	Descrição	Obrig.	Tipo	Observações
method	Nome da função	sim	AN50	regenPayByLink
api.username	User name da API	sim	AN100	
api.password	Password da API	sim	AN20	
token	Token do pedido de pagamento	sim	AN50	
payByLink.paymentDeadline	Data limite de pagamento	sim	Date	dd-mm-aaaa

Exemplo (POST em formato JSON):

```
{
```

```
"method": "regenPayByLink",
"api": {
  "username": "999999999",
  "password": "e6c5a00baecf"
},
"token": "04fb996b2df94b0c6042656559f46a8195ca8065",
"payByLink": {
  "paymentDeadline": "20-12-2025"
}
}
```

Resposta da função “*regenPayByLink*” em caso de sucesso (formato JSON):

```
{
  "result": {
    "code": "00000000",
    "message": ""
  },
  "payByLink": {
    "url": "https://pagamentos.sandbox.reduniq.pt/pay-by-link/339933/...",
    "qrCode": {
      "mimetype": "image/png",
      "base64": "iVBORw0KGgoAAAANSUhEUgAAAFACAIAAABC8..."
    },
    "status": "1"
  },
  "token": "1a3f5ce289818ed8cd66055d705ec5f7d84718c9"
}
```

Resposta da função “*regenPayByLink*” em caso de erro (formato JSON):

```
{
  "result": {
    "code": "00900107",
    "message": "This payment request cannot be renewed."
  },
  "payByLink": {
    "url": null,
    "qrCode": {
      "mimetype": null,
      "base64": null
    },
    "status": null
  },
  "token": ""
}
```

Códigos e mensagens

Tipos de código de resposta:

Código	Descrição
0xxxxxxx	Códigos de resposta do <i>gateway de pagamentos</i>
1xxxxxxx	Códigos de resposta do Reduniq E-Commerce
2xxxxxxx	Códigos de resposta do PayPal (SOAP)
3xxxxxxx	Códigos de resposta do AMEX (3C iPage)
6xxxxxxx	Códigos de resposta do PayPal (REST)
5xxxxxxxxx	Códigos de resposta do Pagamento de serviços (DPG)
7xxxxxxxxx	Códigos de resposta do Cartão de crédito (DPG)
8xxxxxxxxx	Códigos de resposta do MB WAY (DPG)
9xxxxxxxxx	Códigos de resposta do Cartão de crédito (SPG)
10xxxxxxxxx	Códigos de resposta do MB WAY (SPG)
11xxxxxxxxx	Códigos de resposta do Pagamento de serviços (SPG)
12xxxxxxxxx	Códigos de resposta do Parcela Já
13xxxxxxxxx	Códigos de resposta do Cartão de crédito (Cybersource)
14xxxxxxxxx	Códigos de resposta do Google Pay (Cybersource)
15xxxxxxxxx	Códigos de resposta do Apple Pay (Cybersource)
16xxxxxxxxx	Códigos de resposta do PIX (Braza)
17xxxxxxxxx	Códigos de resposta do Click to Pay (Cybersource)
18xxxxxxxxx	Códigos de resposta do Samsung Pay (Cybersource)

Códigos de resposta do *gateway de pagamentos*:

Código	Mensagem
00000000	Success
00100001	Authentication failed
00100002	This merchant has no associated payment solutions
00100003	No response / Timeout / Authentication failed: doWebPayment (Reduniq E-commerce)
00100004	No response / Timeout / Authentication failed: SetExpressCheckout (PayPal)
00100005	No response / Timeout / Authentication failed: getWebPaymentDetails (Reduniq E-commerce)
00100006	No response / Timeout / Authentication failed: DoExpressCheckoutPayment (PayPal)
00100007	Invalid token
00100008	The payment process has not yet started
00100009	The payment process is in progress
00100010	Invalid transaction
00100011	No response / Timeout / Authentication failed: doCapture (Reduniq E-commerce)
00100012	No response / Timeout / Authentication failed: doRefund (PayPal)
00100013	No response / Timeout / Authentication failed: GetExpressCheckoutDetails (PayPal)
00100014	No response / Timeout / Authentication failed: doCapture (PayPal)
00100015	No response / Timeout / Authentication failed: doRefund (Reduniq E-commerce)
00100016	Service not implemented for this payment solution
00100017	Invalid service
00100028	No response / Timeout / Authentication failed: InitialiseService (3C iPage)
00100029	Error getting transaction result (3C iPage)
00100030	No response / Timeout / Authentication failed: RequestAuthorised (3C iPage)

00100031	No response / Timeout / Authentication failed: ReverseByTxID (3C iPage)
00100032	No response / Timeout / Authentication failed: GetStatusByTxID (3C iPage)
00100033	No response / Timeout / Authentication failed: Payment/create (PayPal REST API)
00100034	No response / Timeout / Authentication failed: Payment/execute (PayPal REST API)
00100035	No response / Timeout / Authentication failed: Authorization/capture (PayPal REST API)
00100036	Invalid transaction action
00100037	No response / Timeout / Authentication failed: Prepare the checkout (DGP)
00100038	No response / Timeout / Authentication failed: Get the payment status (DPG)
00100039	No response / Timeout / Authentication failed: Capture an authorization (DPG)
00100040	No response / Timeout / Authentication failed: Reverse a payment (DPG)
00100041	No response / Timeout / Authentication failed: Refund a payment (DPG)
00100042	Invalid payment solution
00100043	No response / Timeout / Authentication failed: Prepare the checkout (SGP)
00100044	No response / Timeout / Authentication failed: Get the payment status (SPG)
00100045	No response / Timeout / Authentication failed: Capture an authorization (SPG)
00100046	No response / Timeout / Authentication failed: Reverse a payment (SPG)
00100047	No response / Timeout / Authentication failed: Refund a payment (SPG)
00100048	Invalid payment deadline
00100049	No response from function or service
00100050	No response / Timeout / Authentication failed: Authorization/get (PayPal REST API)
00100051	No response / Timeout / Authentication failed: RequestNoCardReadByTxID (3C iPage)
00100052	No response / Timeout / Authentication failed: doReset (REDUNIQ E-commerce)
00100053	Invalid refund amount (greater than captured amount)
00100054	No response / Timeout / Authentication failed: Get public token (Parcela Já)
00100055	No response / Timeout / Authentication failed: Get the payment status (Parcela Já)
00100056	There are no payment solutions for this type of payment (action/amount)
00100057	Invalid operation for this transaction
00100058	No response / Timeout / Authentication failed: Capture an authorization (Cybersource)
00100059	No response / Timeout / Authentication failed: Void a payment/capture/refund (Cybersource)
00100060	No response / Timeout / Authentication failed: Refund a payment/capture (Cybersource)
00100061	No response / Timeout / Authentication failed: Create payment token (Cybersource)
00100062	Invalid operation for this payment solution
00100063	Invalid operation for this API version
00100064	Invalid mode for this payment solution
00100065	Invalid action for this payment solution
00100066	Authentication failed (3DS)
00100067	No permissions for this feature
00100068	Invalid transaction Id: Get the payment status (DPG)
00200001	Internal error: Error reading information from the merchant
00200002	Internal error: Error creating transaction
00200003	Internal error: Error reading the transaction result
00200004	Internal error: Error creating payment token
00200005	Internal error: Error creating recurring payment
00200006	Internal error: Error updating recurring payment

00200007	Internal error: Error updating payment token
00200008	Internal error: Error creating PIX
00300000	Invalid parameter: payment.amount
00300001	Invalid parameter: payment.action
00300002	Invalid parameter: payment.description
00300003	Invalid parameter: order.ref
00300004	Invalid parameter: order.amount
00300005	Invalid parameter: order.taxes
00300006	Invalid parameter: order.date
00300007	Invalid parameter: buyer.firstName
00300008	Invalid parameter: buyer.lastName
00300009	Invalid parameter: buyer.email
00300010	Invalid parameter: buyer.shipping.name
00300011	Invalid parameter: buyer.shipping.street1
00300012	Invalid parameter: buyer.shipping.street2
00300013	Invalid parameter: buyer.shipping.city
00300014	Invalid parameter: buyer.shipping.zipCode
00300015	Invalid parameter: buyer.shipping.country
00300016	Invalid parameter: buyer.shipping.phone
00300017	Invalid parameter: returnUrlOk
00300018	Invalid parameter: returnUrlError
00300019	Invalid parameter: notificationUrl
00300020	Invalid parameter: languageCode
00300021	Invalid parameter: transaction.id
00300022	Invalid parameter: comment
00300023	Invalid parameter: buyer.shipping.amount
00300024	Invalid parameter: payment.amount (do not match order amount + order taxes + shipping amount)
00300025	Invalid parameter: order.shipping
00300026	Invalid parameter: buyer.shipping.state
00300027	Invalid parameter: order.shipping (shipping = 0 and shipping.amount > 0)
00300028	Invalid parameter: buyer.shipping (name, street1, city, zipCode and country are mandatory if shipping=1)
00300029	Invalid parameter: buyer.phone
00300030	Invalid parameter: payment.amount (for this payment method, the amount to be returned must be equal to the transaction amount)
00300031	Invalid parameter: buyer.billing.street1
00300032	Invalid parameter: buyer.billing.street2
00300033	Invalid parameter: buyer.billing.city
00300034	Invalid parameter: buyer.billing.state
00300035	Invalid parameter: buyer.billing.zipCode
00300036	Invalid parameter: buyer.billing.country
00300037	Invalid parameter: payment.description
00300038	Invalid parameter: card.number
00300039	Invalid parameter: card.expMonth
00300040	Invalid parameter: card.expYear

00300041	Invalid parameter: card.secCode
00300042	Invalid parameter: recurring.startDate
00300043	Invalid parameter: recurring.endDate
00300044	Invalid parameter: recurring.intervalUnit
00300045	Invalid parameter: recurring.intervalCount
00300046	Invalid parameter: payToken.id
00300047	Invalid parameter: recurring.startDate >= recurring.endDate
00300048	Invalid parameter: recurring.id
00300049	Invalid parameter: payToken.id (expired card)
00300050	Invalid parameter: card.expYear/card.expMonth (expired card)
00300051	Invalid parameter: recurring.endDate (before the last payment date)
00300052	Invalid parameter: payToken.action
00300053	Invalid parameter: recurring.action
00300054	Invalid parameter: payToken.ref
00300055	Invalid parameter: payToken.id (there are active recurring payments associated with this token)
00300056	Invalid parameter: buyer.billing.tin
00300057	Invalid parameter: targetOriginUrl
00300058	Invalid parameter: payToken.shared
00300059	Invalid parameter: payToken.sharedKey
00300060	Invalid parameters: payToken.id/payToken.sharedKey (provide only one of these parameters)
00300061	Invalid parameter: reference
00300062	Invalid parameter: buyer.phone/buyer.email (at least one of these parameters is mandatory)
00300063	Invalid parameter: authentication.step
00300064	Invalid parameter: authentication.returnUrl
00300065	Invalid parameter: authentication.referenceId
00300066	Invalid parameter: authentication.transactionId
00300067	Invalid parameter: authentication.acsWindowSize
00300068	Invalid parameter: filter.startDate
00300069	Invalid parameter: filter.endDate
00300070	Invalid parameter: filter.orderRef
00300071	Invalid parameter: filter.transactionId
00300072	Invalid parameter: filter.solution
00300073	Invalid parameter: filter.status
00300074	Invalid parameter: offset
00300075	Invalid parameter: limit (1 to 2000 records)
00400000	Invalid parameter: order.details.name
00400001	Invalid parameter: order.details.amount
00400002	Invalid parameter: order.details.quantity
00400003	Invalid parameter: order.details.tax
00400004	Invalid parameter: order.details (exceeds the limit of 100 items)
00400005	Invalid parameter: order.details (item amounts do not match order amounts + order taxes)
00400006	Invalid parameter: order.details.category
00500000	Invalid parameter: privateData.key

00500001	Invalid parameter: privateData.value
00500002	Invalid parameter: privateData (exceeds the limit of 100 items)
0600000	Payment canceled by the user
0600001	Payment canceled by the merchant
00700000	Timeout: Redirect to the payment page
00700001	Timeout: In selecting payment method
00700002	Timeout: Finalizing payment
00700003	Timeout: Confirming payment
00800000	Waiting for confirmation of non-instant payment
00800001	Waiting for confirmation of a pending transaction
00900104	An error occurred
00900105	Invalid token
00900106	This payment request cannot be canceled
00900107	This payment request cannot be renewed
00900200	Invalid name
00900201	Invalid surname
00900202	Invalid email
00900203	Invalid phone number
00900204	Invalid address
00900205	Invalid zip code
00900206	Invalid locality
00900207	Invalid country
00900208	Invalid description
00900209	Invalid reference (entity)
00900210	Invalid reference
00900211	Invalid Taxpayer Number
00900212	Invalid payment deadline
00900213	Invalid amount to be paid
00900214	Invalid notification URL
00900215	Invalid mobile number (SMS)
00900216	Sending the payment URL via SMS and WhatsApp is not permitted
00900217	Invalid mobile number (WhatsApp)
00900218	Invalid number of days (validity / asynchronous payment)
00900219	Invalid country (ISO 3166-1)
00900220	Invalid start (days)
00900221	Invalid amount (recurring payment)
00900222	Invalid frequency
00900223	Invalid number of occurrences
00900224	Invalid source URL

Gestão de timeouts

O *gateway* de pagamentos implementa três *timeouts*:

- Após a chamada da função “*initPayment*”, o *token* devolvido é válido por um período de 60 minutos. Ou seja, o tempo decorrido entre a criação do *token* e o redireccionamento do cliente para a página do *gateway*, não pode exceder os 60 minutos.
No modo *lightbox*, o *urlPath* devolvido é válido por 60 minutos.
- Na página do *gateway*, o cliente dispõe de 15 minutos para seleccionar o modo de pagamento pretendido.
- Todo o processo de pagamento, após seleção do modo de pagamento, não deve exceder os 45 minutos.

Cada solução de pagamentos implementa os seus próprios *timeouts*. Para mais informações, consultar os respetivos guias de integração.